

Insert Using persist() method

In Hibernate, inserting data into the database is typically achieved using session operations like `persist()`, `save()`, or `saveOrUpdate()`. This process involves creating an entity, starting a transaction, and using the session object to persist the entity into the database.

Steps to Insert Data Using Hibernate:

- 1. Configure Hibernate:**
 - Create a Configuration object to configure Hibernate with properties and mappings.
 - Load the `hibernate.cfg.xml` file with `config.configure()`.
- 2. Create a SessionFactory and Session:**
 - SessionFactory is a factory for creating Session objects.
 - Open a session using `sessionFactory.openSession()`.
- 3. Begin a Transaction:**
 - Always begin a transaction before performing any database operation.
 - Use `session.beginTransaction()` to start a transaction.
- 4. Create an Entity Object:**
 - Instantiate the entity object (e.g., Student) and set its properties.
- 5. Insert Entity Using `session.persist()`:**
 - Use `session.persist()` to insert the entity into the database.
 - Commit the transaction using `transaction.commit()`.
- 6. Handle Exceptions:**
 - Catch exceptions such as `EntityExistsException`, `ConstraintViolationException`, and `HibernateException`.
- 7. Close Resources:**
 - Use `session.close()` and `sessionFactory.close()` in the finally block.

Exceptions during Insert Operations in Hibernate

- 1. EntityExistsException:**
 - Thrown when trying to persist an entity that already exists in the database.

```
catch (EntityExistsException e) {  
    System.out.println("Entity already exists: " + e.getMessage());  
}
```

2. ConstraintViolationException:

- Thrown when database integrity constraints (like unique constraints) are violated.

```
catch (ConstraintViolationException e) {  
    System.out.println("Constraint violation: " + e.getMessage());  
}
```

3. HibernateException:

- A general exception that indicates a problem with the Hibernate framework.

```
catch (HibernateException e) {  
    System.out.println("Hibernate exception: " + e.getMessage());  
}
```

4. Exception:

- Catches any other unexpected exceptions.

```
catch (Exception e) {  
    System.out.println("Unexpected exception: " + e.getMessage());  
}
```

persist() vs save() in Hibernate

1. persist() Method:

- **Purpose:** Used to insert an entity into the database if it does not already exist.
- **Return Type:** void (does not return the identifier).
- **Behavior:**

- Makes the given entity persistent and schedules it for insertion during flush/commit.
- Does **not** guarantee the immediate insertion.
- **Exception Handling:** Throws EntityExistsException if the entity already exists in the database.

```
session.persist(student);
```

2. save() Method:

- **Purpose:** Used to insert an entity into the database.
- **Return Type:** Returns the generated identifier (primary key) of the entity.
- **Behavior:**
 - Guarantees the immediate insertion of the entity.
 - The generated identifier is immediately available after the call.
- **Exception Handling:** Unlike persist(), save() will insert the entity, even if it is in the transient state.

```
Serializable id = session.save(student);  
System.out.println("Student ID: " + id);
```

Example of save() and persist() Methods in Hibernate

Using persist():

```
Transaction transaction = session.beginTransaction();  
Student student = new Student();  
student.setId(134);  
student.setName("Arjun");  
student.setCity("Ballia");  
  
session.persist(student); // Insert the student entity  
transaction.commit();    // Commit the transaction
```

Using save():

```

Transaction transaction = session.beginTransaction();
Student student = new Student();
student.setName("Arjun");
student.setCity("Ballia");

Serializable id = session.save(student); // Insert the student entity and return the ID
transaction.commit();
System.out.println("Student ID: " + id); // Print the generated ID

```

Key Differences Between persist() and save():

Aspect	persist()	save()
Return Type	void	Returns Serializable (primary key ID)
Entity State	Transient -> Persistent	Transient -> Persistent
Insert Guarantee	Not immediate, scheduled at flush	Immediate upon calling
Exception Handling	Throws EntityExistsException	No exception for existing entities
Recommended Usage	For standardized JPA usage	When you need the generated ID

Industry Standard Codes:

Main.java

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
    public static void main(String[] args) {
        Configuration config = null;
        SessionFactory sessionFactory = null;
        Session session = null;
        Transaction transaction = null;
        boolean flag = false;
        try {
            config = new Configuration();
            config.configure();
            sessionFactory = config.buildSessionFactory();
            session = sessionFactory.openSession();
            Student student = new Student();
            student.setId(134);
            student.setName("Arjun");
            student.setCity("Ballia");
            transaction = session.beginTransaction();
            session.persist(student);
            transaction.commit();
            System.out.println("Transaction committed Succesfully");
            flag = true;
        } catch (EntityExistsException e) {
            System.out.println("Entity already exists: " + e.getMessage());
        } catch (ConstraintViolationException e) {
```

```

        System.out.println("Constraint violation: " + e.getMessage());
    } catch (HibernateException e) {
        System.out.println("Hibernate exception: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected exception: " + e.getMessage());
    } finally {
        if(!flag)
        {
            if (transaction != null) {
                transaction.rollback();
                System.out.println("Transaction rolled back successfully");
            }
        }
        if (session != null) {
            session.close();
        }
        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
}
}

```

Student.java

```

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="StudentTable")
public class Student {
    @Id

```

```
@Column(name="sid")  
    private int id;
```

```
@Column(name="sname")  
    private String name;
```

```
@Column(name="scity")  
    private String city;
```

```
    public Student() {  
        System.out.println("Zero param constructor");  
    }
```

```
    public int getId() {  
        return id;  
    }
```

```
    public void setId(int id) {  
        this.id = id;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

```
    public String getCity() {  
        return city;  
    }
```

```
    public void setCity(String city) {  
        this.city = city;  
    }
```

```
}
```

Result:

```
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionPr
Zero param constructor
Hibernate:
    insert
    into
        StudentTable
        (scity, sname, sid)
    values
        (?, ?, ?)
Transaction committed Successfully
Sept 07, 2024 3:36:55 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/telusko_db]
```

```
mysql> select * from studenttable;
+-----+-----+-----+
| sid   | scity  | sname  |
+-----+-----+-----+
| 13    | Ballia | Arjun  |
| 123   | Ballia | Shiva  |
| 134   | Ballia | Arjun  |
| 1233  | Ballia | Arjun  |
| 1345  | Ballia | Shiva  |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Final Notes:

- Always handle exceptions gracefully to ensure the transaction is rolled back when necessary.
- Use `persist()` when following JPA specifications and `save()` when you need to get the generated identifier immediately.